

Set up notifications using the Amazon Simple Queue Service workflow

GET STARTED

- Welcome to SP-API Documentation
- What is the Selling Partner API?
- Selling Partner API Onboarding Overview >

- Terminology
- Frequently Asked Questions >

CHANGELOG

- SP-API Release Notes
- SP-API Deprecation Schedule

Set up notifications using the Amazon Simple Queue Service workflow

Learn how to set up your system to receive notifications using the Amazon Simple Queue Service workflow.

Learn how to set up your system to receive the following notifications using the Amazon Simple Queue Service workflow:

- `ACCOUNT_STATUS_CHANGED` . Sent when the Account Status changes for the developers subscribed selling partner/marketplace pairs. A notification is published when the selling partner's account status changes between `NORMAL` , `AT_RISK` , and `DEACTIVATED` .
- `ANY_OFFER_CHANGED` . Sent when there is a listing change for any of the top 20 offers, by condition (new or used), or if the external price (the price from other retailers) changes for an item listed by the seller.
- `B2B_ANY_OFFER_CHANGED` . Sent when there is a change in any of the top 20 B2B offers, in the form of any price change (either single unit or quantity discount tier prices) for an item listed by the seller.
- `DETAIL_PAGE_TRAFFIC_EVENT` : Sent at the beginning of every hour. This notification shares traffic data at an ASIN level and includes data for the previous hour, as well as any delayed data from up to 24 hours earlier. Each notification can include multiple ASINs, and a selling partner can expect to receive multiple notifications every hour.
- `EXTERNAL_FULFILLMENT_SHIPMENT_STATUS_CHANGE` : Sent when there is a change in a

SP-API Product Metadata Updates

DIRECTORIES

SP-API Models

SP-API Seller Use Cases

SP-API Vendor Use Cases

Seller Central URLs

Vendor Central URLs

REGISTRATION

SP-API Registration Overview >

Developer Registration Request Status

Third-Party Provider Registration >

Service Provider Registration and Role Approval Process Overview

Website Guidelines for Public Developers and Service Providers

User Permissions for Service Providers

Register your Application

Update your Application Information

Rotate your Application's LWA Credentials >

View your Application Information and Credentials

Learn How Sellers Authorize Service Providers

Selling Partner API Roles >

Policies and Agreements

About Facial Data

AUTHORIZATION

Authorize Applications >

Renew Authorizations

Revoke Authorizations

Authorization Limits

shipment for the External Fulfillment API.

- [FBA_INVENTORY_AVAILABILITY_CHANGES](#) . Sent when there is a change in the Fulfillment By Amazon (FBA) inventory quantities. This notification includes a snapshot of the FBA inventory in all eligible marketplaces in a particular region.
- [FBA_OUTBOUND_SHIPMENT_STATUS](#) . Sent when we create or cancel a Fulfillment by Amazon shipment for a seller.
- [FEE_PROMOTION](#) . Sent when a promotion becomes active.
- [FEED_PROCESSING_FINISHED](#) . Sent when any feed submitted using the Selling Partner API for Feeds reaches a feed processing status of `DONE` , `CANCELLED` , or `FATAL` .
- [FULFILLMENT_ORDER_STATUS](#) . Sent when there is a change in the status of a Multi-Channel Fulfillment order.
- [ITEM_INVENTORY_EVENT_CHANGE](#) : Sent at the beginning of every hour. This notification shares inventory data at an ASIN level, and includes data for the previous hour, as well as any delayed data from up to 24 hours earlier. Each notification can include multiple ASINs, and a selling partner can expect to receive multiple notifications every hour.
- [ITEM_SALES_EVENT_CHANGE](#) : Sent at the beginning of every hour. This notification shares sales data at an ASIN level and includes data for the previous hour, as well as any delayed data from up to 24 hours earlier. Each notification can include multiple ASINs, and a selling partner can expect to receive multiple notifications every hour.
- [ORDER_CHANGE](#) : Sent when there is an important change in the order. Important changes include order status changes and buyer requested cancelations. Note that `marketplaceIds` is not a supported filter for [ORDER_CHANGE](#) . If you want to include `marketplaceIds` , you must use [ANY_OFFER_CHANGED](#) .
- [PRICING_HEALTH](#) . Sent when a seller offer is ineligible to be the Featured Offer (Buy

Special Authorization Types

> Box offer) because of an uncompetitive price.

INTEGRATION

Building Robust Amazon SP-API Applications

SP-API SDKs

- >
- **REPORT_PROCESSING_FINISHED** : Sent when any report that you have requested using the Selling Partner API for Reports reaches a report processing status of **DONE** , **CANCELLED** , or **FATAL** .
 - **TRANSACTION_UPDATE** : Sent whenever there is a new transaction posted to the seller's account.

**Warning**

To receive any other notification type, refer to [Set up notifications \(Amazon EventBridge workflow\)](#).

Prerequisites

To complete this tutorial you need:

- Authorization from the selling partners for whom you are making calls. Refer to [Authorizing Selling Partner API applications](#) for more information.
- An AWS Account. If you are not already an AWS customer, you can create a free AWS account. For more information, refer to [AWS Free Tier](#).
- An Amazon Simple Queue Service (SQS) queue. For information about setting up an SQS queue, refer to [Amazon Simple Queue Service](#).

Delete an Application From Your Developer Account

CALLING THE SP-API

Configuration Details

>

Call Structure

>

Development Tools

>

Performance Management

>

Step 1. Grant the Selling Partner API permission to write to your SQS queue

To receive notifications to your SQS queue, you must grant the Selling Partner API permission to write to the queue. To grant permissions to your SQS queue, you can upload an AWS

CloudFormation template or you can configure permissions manually by using the Amazon SQS console. For instructions, refer to [Tutorial: Grant the SP-API Permission to an Amazon SQS Queue](#).

Step 2. Grant the Selling Partner API permission to

SOLUTION PROVIDER PORTAL

- Manage Users in Solution Provider Portal
- Manage Your Business And Contact information in Solution Provider Portal
- Update Your Developer Profile in Solution Provider Portal
- Unify Your Developer Accounts
- Verify Your Identity in Solution Provider Portal
- API Usage Metrics in Solution Provider Portal

TUTORIALS

- Tutorial: Subscribe to the ORDER_CHANGE Notification
- Tutorial: Test Selling Partner API Endpoints
- Tutorial: Automate your SP-API Calls Using a C# SDK
- Tutorial: Automate your SP-API Calls Using a Java SDK
- Tutorial: Automate your SP-API Calls Using a JavaScript SDK for Node.js
- Tutorial: Automate your SP-API Calls Using a Python SDK
- Tutorial: Automate your SP-API Calls using a prebuilt C# SDK
- Tutorial: Automate your SP-API Calls using prebuilt Java SDK
- Tutorial: Automate your SP-API Calls using a prebuilt JavaScript SDK
- Tutorial: Automate your SP-API Calls using prebuilt PHP SDK
- Tutorial: Automate your SP-API calls using a prebuilt Python SDK
- Tutorial: Authorize Multiple Vendor Central Accounts with a Single SP-API Application
- Tutorial: Retrieve Merchant Shipping Templates
- Tutorial: Grant the SP-API Permission to an Amazon SQS Queue
- Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier

TROUBLESHOOTING

- Troubleshoot SP-API Errors
- URL Encoding
- Resolve Common HTTP and Authorization Error Codes
- External Fulfillment Errors 
- Listings Items API Issues Troubleshooting 

SELLING PARTNER APPSTORE

- What is the Selling Partner Appstore?
- List Your App on the Selling Partner Appstore

access your server-side encryption key

If you're using server-side encryption (SSE), you must give the Selling Partner API access to your key by using the [AWS Key Management Service \(KMS\)](#). Consider using the following policy.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS":
          "arn:aws:iam::437568002678:root"
      },
      "Action": [
        "kms:GenerateDataKey",
        "kms:Decrypt"
      ],
      "Resource": "*"
    }
  ]
}
```

For more information, refer to [Key management](#).

Step 3. Create a destination

Call the `createDestination` operation to create an Amazon Simple Queue Service (SQS) destination.

Parameter	Description	Required
resourceSpecification	The information required to create an SQS destination. This includes the <code>sqs:arn</code> value that you got from Step 1. Grant Selling Partner API permission to write to your SQS queue . Type: object	Yes.
name	A name that you specify to help you identify this destination. Type: string	Yes

Edit Your Appstore Listing	
Check Listing Status	
Appstore Ratings and Reviews	
Press Releases and Promotions	
SERVICE PROVIDER NETWORK	
List Your Service	
SECURITY AND COMPLIANCE	
SP-API Security and Compliance Overview	
Amazon Selling Partner API Guard Implementation Guide	>
VAT Calculation Service	>
Amazon Seller Data Access	
Best Practices	>



Note

Calling the `createDestination` operation does not require authorization from any selling partner. In this respect, this operation is a "grantless" operation and has a different authorization model from most other Selling Partner API operations. For more information, refer to [Grantless operations](#).

Save the `destinationId` value as input for [Step 3. Create a subscription](#).

Step 4. Create a subscription

Create a subscription to a notification type, to be delivered to the destination that you created in the previous step.



Note

We allow multiple subscriptions with different payload versions per application, notification type, and party (seller/vendor). For supported payload versions of different notification types, refer to `notificationType`.

Call the `createSubscription` operation, passing the following parameters:

Path parameters:

Parameter	Description	Required
<code>notificationType</code>	Information about the notification type and payload version to which you want to subscribe. Possible values: <code>ACCOUNT_STATUS_CHANGED</code> , <code>ANY_OFFER_CHANGED</code> , <code>B2B_ANY_OFFER_CHANGED</code> , <code>DETAIL_PAGE_TRAFFIC_EVENT</code> , <code>FBA_INVENTORY_AVAILA</code>	Yes

Parameter	Description	Required
A+ CONTENT API A+ Content API > A+ Content Examples AMAZON WAREHOUSING AND DISTRIBUTION API Amazon Warehousing and Distribution API > Amazon Warehousing and Distribution API v2024-05-09 Reference	BILITY_CHANGE S , FBA_OUTBOUN D_SHIPMENT_STA TUS , FEE_PROMO TION , FEED_PRO CESSING_FINISH ED , FULFILLMEN T_ORDER_STATU S , ITEM_INVENT ORY_EVENT_CHAN GE , ITEM_SALES _EVENT_CHANGE , ORDER_CHANGE , PRICING_HEALT H , REPORT_PRO CESSING_FINISHE D . Refer to noti ficationType Type: string	

APP INTEGRATIONS API

App Integrations API



Body parameters:

Parameter	Description	Required
payloadV ersion	The version of the payload object to be used in the notification. For supported payloadVersion for different notification types, refer to no tificationTyp e . Type: string	Yes
APPLICATION MANAGEMENT API Application Management API > CATALOG ITEMS API Catalog Items API > Catalog Items API Rate Limits Catalog Items API v2022-04-01 Reference	The identifier for the destination where notifications will be delivered. Use the value you saved in Step 2. Create a destination. Type: string	Yes
CUSTOMER FEEDBACK API Customer Feedback API > Customer Feedback API Rate Limits	Additional information passed to the subscription to control how	No

DATA KIOSK

Data Kiosk API >

Data Kiosk Schema Explorer User Guide

Data Kiosk Query Processing Finished Notification

Building Data Kiosk workflows guide

Vendor Analytics Dataset Use Case Guide

Schedule Data Kiosk Queries

DELIVERY BY AMAZON API

Delivery by Amazon API >

EASY SHIP API

Easy Ship API >

EXTERNAL FULFILLMENT

External Fulfillment Inventory API >

External Fulfillment Returns API >

External Fulfillment Shipping API >

External Fulfillment Errors

External Fulfillment APIs Rate Limits

FULFILLMENT BY AMAZON (FBA)

FBA Inventory API >

FBA Inventory Dynamic Sandbox Guide

FEEDS API

Parameter	Description	Required
	notifications are processed. For example, you can use an <code>eventFilter</code> to customize your subscription to send notifications for only the specified <code>marketplaceId</code>, or select the aggregation time period at which to send notifications (for example, limit to one notification every five minutes for high frequency notifications). The specific features available vary depending on the <code>notificationType</code>. Note: The <code>eventFilter</code> is currently only supported by the <code>ANY_OFFER_CHANGED</code> and <code>ORDER_CHANGE</code> notificationTypes. Alternatively, you can use a <code>filterExpression</code> to define a CEL expression that filters events based on payload fields. The <code>filterExpression</code> is supported by additional notification types. For more information, refer to Filter notification subscriptions.	

Feeds API	>
Feed Type Values	>
Feeds JSON Schemas	↗
Feeds API Best Practices	
Feeds API FAQ	
Feeds API Rate Limits	
FINANCES	
Finances API	>
Transfers API	>
FULLFILLMENT INBOUND API	
Fulfillment Inbound API	>
Fulfillment Inbound API FAQ	
Migrating Fulfillment Inbound workflows	
Fulfillment Inbound API Rate Limits	
FULLFILLMENT OUTBOUND API	
Fulfillment Outbound Dynamic Sandbox Guide	
Fulfillment Outbound API	>

Parameter	Description	Required
	Type: <code>ProcessingDirective</code>	

Processing notifications from your queue

There are several important properties of Amazon SQS queues that you must understand in order to process notifications properly:

- **Selling Partner API does not support delivery to FIFO queues.** You must use Amazon SQS standard queues to receive notifications.
- **Selling Partner API standard queues do not guarantee that notifications will be received in the order they were sent.** Standard queues provide best-effort ordering, which means that notifications are generally delivered in the same order as they're sent. However, occasionally, more than one copy of a notification might be delivered out of order. Therefore, you must design your application to accept notifications in any order.
- **Amazon SQS standard queue notifications might be delivered more than once.** Amazon SQS stores copies of your notifications on multiple servers for redundancy and high availability. On rare occasions, one of the servers that stores a copy of a notification might be unavailable when you receive or delete a notification. If this scenario occurs, the copy of the notification isn't deleted on that unavailable server, and you might get that notification copy again when you receive notifications. Therefore, you must design your application to accept multiple copies of any given notification.
- You can determine if a notification is a duplicate of a notification you have already received by looking at the `notificationId` property of the notification. You can find the `notificationId` property in the `NotificationMetaData` object of the notification.

For more information about processing notifications from Amazon SQS queues, refer to the [Amazon Simple Queue Service Developer](#)

[Guide](#) and the [Amazon Simple Queue Service API Reference](#) in the [AWS Documentation](#) portal.

Processing Directives

A processing directive is an optional parameter that you can provide when you call the `createSubscription` operation. It can be used to change the behavior of a subscription for certain `notificationTypes`.

Use a `processingDirective` to pass additional information to the subscription to control the processing of notifications. You can use one of the following approaches:

- eventFilter** : Customize your subscription to send notifications for only the specified `marketplaceId`, or select the aggregation time period at which to send notifications (for example, limit to one notification every five minutes for high frequency notifications). The `eventFilter` is currently only supported for use with the `ANY_OFFER_CHANGED` and `ORDER_CHANGE` notification types. Usage with an unsupported notification type results in a request failure with an HTTP 400 response.
- filterExpression** : Define a [CEL](#) expression that evaluates against each event's payload and returns only the events that match your criteria. The `filterExpression` is supported by additional notification types. For the full list of eligible types, supported operators, and examples, refer to [Filter notification subscriptions](#).



Note

You cannot use `eventFilter` and `filterExpression` together in the same subscription. If you need aggregation and filtering, continue to use `eventFilter`.

The following `eventFilter` values are supported:

Name	Description
<code>eventFilterType</code>	An <code>eventFilterType</code> value that is supported by the specific <code>notificationType</code> . This is used by

Fulfillment Outbound API Rate Limits

MCF Best Practices

INVOICING

Invoices API

Invoices API FAQ

LISTINGS

Listings Items API

		Name	Description
Listings Items API Rate Limits Listings Restrictions API > Listings Restrictions API Rate Limits Manage Product Listings with the Selling Partner API Building Listings Management Workflows Guide Understanding Amazon listing status and seller-fulfilled inventory management Listings Management Workflow Migration > Manage Amazon Haul, Advanced Multiple-Offer, and Multiple-Fulfillment Use Cases Listings APIs FAQ Product Type Definitions API > Product Type Definitions API Rate Limits MERCHANT FULFILLMENT API Merchant Fulfillment API >			the subscription service to determine the type of event filter. Example: <code>"eventFilterType": "ANY_OFFER_CHANGED"</code> Required.
		aggregationSettings	A container to support aggregated filtering of notifications. Supports using an <code>aggregationTimePeriod</code> to limit (filter) the sending of notifications to a five or ten minute frequency. Example: <code>"aggregationSettings": { "aggregationTimePeriod": "FiveMinutes" }</code> Optional.
		marketplaceIds	A list of marketplace identifiers to subscribe to. To receive notifications for every marketplace, do not provide this list. <code>marketplaceIds</code> is not a supported filter for the <code>ORDER_CHANGE</code> notification. If you want to include <code>marketplaceIds</code> , you must use <code>ANY_OFFER_CHANGED</code> . Example: <code>"marketplaceIds": ["ATVPDKIKX0DER", "A2EUQ1WTGCTBG2"]</code> Optional.
		orderChangeTypes	A list of order change types to subscribe to. To receive notifications for every order change type, do not provide this list. Possible values are: <code>BuyerRequestedChange</code> <code>DeliveryTipChange</code> <code>OrderStatusChange</code> Example: <code>"orderChangeTypes": ["BuyerRequestedChange"]</code> Optional.

For more information, refer to [ProcessingDirective](#) in the API reference.

MESSAGING API

Messaging API >

Notification structure

Selling Partner notifications are in JSON format. Each notification contains a `Payload` object, which contains the actionable data of the notification. [Notification Type](#), in combination with `PayloadVersion`, determines the structure of the `Payload` object.

A Selling Partner notification with `NotificationVersion` =1.0 contain the following properties:

Object	Description	Type
<code>NotificationVersion</code>	The notification version. This controls the structure of the notification.	string
<code>NotificationType</code>	The notification type. <code>NotificationType</code> , combined with <code>PayloadVersion</code> , controls the structure of the <code>Payload</code> object.	string
<code>PayloadVersion</code>	The payload version. <code>PayloadVersion</code> , combined with <code>NotificationType</code> , controls the structure of the <code>Payload</code> object. For supported <code>PayloadVersion</code> for different notification types, refer to notificationType .	string
<code>EventTime</code>	The date and time (in UTC) that the event which triggered the notification occurred.	string
<code>Payload</code>	The actionable data of the notification. The structure of the <code>Payload</code> is	JSON object For more information,

NOTIFICATIONS API

Notifications API



Create and Filter Notification Subscriptions

Set up notifications using the Amazon EventBridge workflow

Set up notifications using the Amazon Simple Queue Service workflow

Notification Type Values

Tutorial: Subscribe to the ORDER_CHANGE Notification



ORDERS API

Orders API



Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier



Orders API Migration Guide

Orders API Rate Limits

PRODUCT FEES API

Product Fees API



PRODUCT PRICING API

Product Pricing API



Product Pricing API and Notifications FAQ

Price Adjustment Automation Workflows Guide
Manage automated pricing rules with SP-API

REPLENISHMENT API

Replenishment API



REPORTS API

Reports API



Report Type Values



Reports JSON Schemas



Reports API FAQ

SALES API

Sales API



SELLER WALLET API

Seller Wallet API



Object	Description	Type
	determined by NotificationType, in combination with PayloadVersion.	refer to Notifications .
NotificationMetadata	The notification metadata. This includes the following objects: - ApplicationId - The identifier for the application that uses the notifications. Type = string - SubscriptionId - A unique identifier for the subscription which resulted in this notification. Type = string - PublishTime - The date and time (in UTC) that the notification was sent. Type = string - NotificationId - A unique identifier for this notification instance. Type = string	JSON object

Notification example:

		JSON
Seller Wallet API Rate Limits		<pre>{ "NotificationVersion": "1.0", "NotificationType": "BRANDED_ITEM_CONTENT_CHANGE", "PayloadVersion": "1.0", "EventTime": "2019-03- 20T18:59:30.194Z", "Payload": { "MarketplaceId": "ATVPDKIKX0DER", "BrandName": "Great Brand", "Asin": "B1234567", "AttributesChanged": ["bullet_point", "item_name", "product_description", "main_product_image_locator", "other_product_image_locator_1", "other_product_image_locator_2", "other_product_image_locator_3", "other_product_image_locator_4", "other_product_image_locator_5", "other_product_image_locator_6", "other_product_image_locator_7", "other_product_image_locator_8", "swatch_product_image_locator"] }, "NotificationMetadata": { "ApplicationId": "amzn1.sellerapps.app.f1234566-aaec- 55a6-b123-bcb752069ec5", "SubscriptionId": "93b098e1-c42- 2f45-93a1-78910a6a8369", "PublishTime": "2019-03- 20T18:59:48.768Z", "NotificationId": "8e009934-da2c- 4f9c-9bc7-93f23b7e1f60" } }</pre>
SELLERS API		
Sellers API	>	
SERVICES API		
Services API	>	
SHIPMENT INVOICING API		
Shipment Invoicing API	>	
SHIPPING API		
Shipping API v2 Resources	↗	
Shipping API v1 Reference		
SOLICITATIONS API		
Solicitations API	>	
SUPPLY SOURCES API		
Supply Sources API	>	

Common types

Contains common types that are used by all notifications that are contained in the Notification payload objects.

FulfillmentChannelType

TOKENS API

Tokens API

Set up notifications using the Amazon Simple Queue Service workflow

Indicates whether the item is fulfilled by Amazon or by the seller.

Type: string

FulfillmentChannelType values:

- Amazon
- Merchant

moneyType

Currency type and amount.

The following table shows the properties of the MoneyType object:

Name	Description
amount	The currency amount. Type: number
currency Code	Three-digit currency code. In ISO 4217 format . Type: string

MoneyType

Currency type and amount.

The following table shows the properties of the MoneyType object:

Name	Description
Amount	The currency amount. Type: number
Currency Code	Three-digit currency code. In ISO 4217 format . Type: string

Updated 19 days ago

Set up notifications using the Amazon EventBridge workflow

Notification Type Values →

Did this page help you? Yes No

UPLOADS API

Uploads API

VEHICLES API

Vehicles API

Vehicles API Rate Limits

VENDOR DIRECT FULFILLMENT APIS

Vendor Direct Fulfillment Dynamic Sandbox Guide

Vendor Direct Fulfillment Workflow Guide

SP-API Bill-to-Party Addresses

VENDOR DIRECT FULFILLMENT TRANSACTION STATUS API

Vendor Direct Fulfillment Transaction Status API

VENDOR DIRECT FULFILLMENT INVENTORY API

Vendor Direct Fulfillment Inventory API

VENDOR DIRECT FULFILLMENT ORDERS API

Vendor Direct Fulfillment Orders API

VENDOR DIRECT FULFILLMENT SHIPPING API

Vendor Direct Fulfillment Shipping API

[Policies and Agreements](#)[Privacy notice](#) | [Conditions of use](#)**VENDOR DIRECT FULFILLMENT PAYMENTS API**[Vendor Direct Fulfillment Payments API](#) >**VENDOR RETAIL PROCUREMENT INVOICES API**[Vendor Retail Procurement Invoices API](#) >**VENDOR RETAIL PROCUREMENT ORDERS API**[Vendor Retail Procurement Orders API](#) >**VENDOR RETAIL PROCUREMENT SHIPMENTS API**[Vendor Retail Procurement Shipments API](#) >**VENDOR RETAIL PROCUREMENT TRANSACTION STATUS API**[Vendor Retail Procurement Transaction Status API](#) >